

Dive into Deep Learning (D2L)

Convolutional Neural Networks and LeNet

Joon Soo Yoo, Ph.D.

Spring 2026

Assistant Professor

Defense & Advanced Security (DAS) Lab

Jeonbuk National University (JBNU)

CNN: 이미지를 위한 신경망

오늘의 목표

- MLP가 이미지 처리에 비효율적인 이유를 이해한다.
- Convolution, kernel, channel, padding, stride, pooling의 역할을 구분한다.
- LeNet이 이미지를 어떻게 feature로 바꾸고 classification을 수행하는지 따라간다.
- 마지막에는 `lenet_experiment.py` 코드의 실험 흐름을 해석한다.

수업의 큰 질문

CNN은 왜 “이미지 전체를 한 번에 외우는 모델”이 아니라, “작은 패턴을 반복해서 찾는 모델”인가?

Part 1. CNN의 핵심 아이디어

1. MLP로 이미지를 보면 생기는 문제

- 이미지는 본래 2차원 또는 3차원 구조를 가진다.
- MLP는 이미지를 긴 벡터로 펼쳐서 처리한다.
- 이 과정에서 가까운 픽셀끼리 가진 공간적 관계가 약해진다.

예시

1000 × 1000 이미지에서 모든 입력 픽셀과 모든 출력 픽셀을 연결하면 파라미터 수가 10^{12} 수준까지 커질 수 있다.

$$H_{i,j} = U_{i,j} + \sum_k \sum_l W_{i,j,k,l} X_{k,l}$$

핵심 문제

모든 위치마다 다른 weight를 쓰면 이미지의 반복 구조를 활용하지 못한다.

2. CNN의 세 가지 관찰

이미지 데이터에 대한 좋은 가정

아이디어	의미
Locality	중요한 특징은 대개 작은 국소 영역에서 나타난다.
Translation Invariance	같은 패턴은 위치가 바뀌어도 같은 의미를 가진다.
Hierarchical Representation	edge, texture, part, object처럼 단순한 특징이 복잡한 특징으로 쌓인다.

한 줄 요약

CNN은 이미지의 구조를 버리지 않고, 작은 필터를 여러 위치에 공유해서 적용한다.

3. Translation Invariance: 위치보다 패턴

- 물체가 왼쪽에 있든 오른쪽에 있든 같은 물체로 인식해야 한다.
- 그래서 같은 kernel을 이미지 전체 위치에 반복 적용한다.
- 이것이 weight sharing의 직관이다.

$$H_{i,j} = u + \sum_a \sum_b V_{a,b} X_{i+a,j+b}$$



4. Locality: 전체가 아니라 근처만 본다

- 한 출력 위치를 계산할 때 입력 이미지 전체를 볼 필요가 없다.
- 작은 window 안의 픽셀만 사용하면 edge, corner, texture 같은 국소 패턴을 잡을 수 있다.
- 이 가정을 적용하면 weight 수가 크게 줄어든다.

$$H_{i,j} = u + \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} V_{a,b} X_{i+a,j+b}$$

Convolution의 출발점

같은 작은 weight 묶음(kernel)을 여러 위치에 공유해서 적용한다.

4-1. Locality가 잡는 것: Edge, Corner, Texture

- 작은 window는 단순하지만 중요한 패턴을 감지한다.
- CNN은 이런 기본 패턴들을 조합하여 더 복잡한 의미를 만든다.

Edge (엣지)

픽셀 값이 급격히 변하는 경계
→ 물체의 윤곽선, 경계

Corner (코너)

여러 방향의 edge가 만나는 지점
→ 모서리, 방향 변화

Texture (텍스처)

반복되는 패턴이나 질감
→ 털, 벽, 바닥 무늬

핵심 직관

CNN은 처음부터 고양이를 보는 것이 아니라,
edge → **corner** → **texture** → **object** 순서로 이해한다.

4-2. Edge Detection 예시: 필터가 경계를 찾는다

- 입력 이미지가 단순한 0과 1로 구성되어 있다고 가정하자.
- 가로 방향 필터 $[-1, 1]$ 을 적용하면 픽셀 간의 차이를 계산한다.

입력 (예시)

$$X = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix}$$

필터 (Kernel)

$$K = [-1 \quad 1]$$

연산 결과

$$(1, 1) \rightarrow -1 \cdot 1 + 1 \cdot 1 = 0$$

$$(1, 0) \rightarrow -1 \cdot 1 + 1 \cdot 0 = -1$$

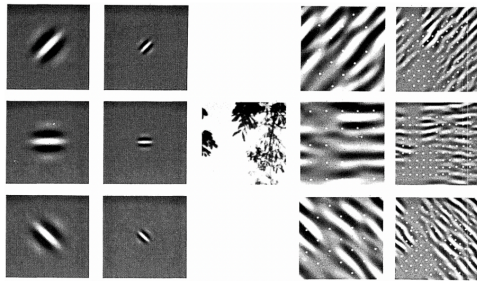
$$(0, 0) \rightarrow -1 \cdot 0 + 1 \cdot 0 = 0$$

5. 계층적 특징 학습

- 초기 layer: edge, line, color 변화
- 중간 layer: texture, corner, shape
- 깊은 layer: object part, class-level feature

직관

얼굴 인식은 한 번에 “얼굴”을 찾는 것이 아니라, 선과 모양을 쌓아 얼굴이라는 복잡한 패턴을 만든다.



Part 2. Convolution 연산

1. 합성곱의 기본 형태

- CNN에서 kernel은 이미지 위를 sliding하면서 각 위치의 feature 값을 계산한다.
- D2L의 구현 관점에서는 엄밀히 말해 cross-correlation을 사용한다.
- 수업에서는 관례적으로 convolution layer라고 부른다.

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

중요한 감각

kernel은 “무엇을 찾을지”를 담고 있고, feature map은 “어디에서 얼마나 강하게 나타났는지”를 담는다.

2. Cross-Correlation 예시

$$X = \begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \end{bmatrix}, \quad K = \begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix}$$

$$Y = \begin{bmatrix} 19 & 25 \\ 37 & 43 \end{bmatrix}$$

계산 방식

같은 크기의 window와 kernel을 element-wise multiply한 뒤 모두 더한다.

3. Convolution을 직접 구현하기

2D Cross-Correlation

```
import torch

def corr2d(X, K):
    h, w = K.shape
    Y = torch.zeros((X.shape[0] - h + 1,
                     X.shape[1] - w + 1))

    for i in range(Y.shape[0]):
        for j in range(Y.shape[1]):
            Y[i, j] = (X[i:i+h, j:j+w] * K).sum()

    return Y
```

코드 해석

두 개의 for loop는 kernel이 이동하는 위치를 나타내고, slicing은 현재 kernel이 보고 있는 작은 영역을 의미한다.

4. 학습 가능한 Convolution Layer

Learnable Kernel and Bias

```
import torch
import torch.nn as nn

class Conv2D(nn.Module):
    def __init__(self, kernel_size):
        super().__init__()
        self.weight = nn.Parameter(torch.rand(kernel_size))
        self.bias = nn.Parameter(torch.zeros(1))

    def forward(self, x):
        return corr2d(x, self.weight) + self.bias
```

Layer가 된다는 뜻

kernel과 bias가 parameter가 되면, backpropagation으로 데이터에 맞는 feature detector를 학습할 수 있다.

5. Edge Detection: 사람이 설계한 kernel

$$K = \begin{bmatrix} 1 & -1 \end{bmatrix}$$

- 인접한 두 픽셀의 차이를 계산한다.
- 값이 변하지 않으면 0에 가깝다.
- 밝기 변화가 있으면 edge로 반응한다.

$$1 \rightarrow 0 : +1 \quad 0 \rightarrow 1 : -1$$

직관

이 kernel은 수직 방향 경계를 찾는 간단한 detector이다.

6. Edge Kernel도 학습할 수 있다

Learning an Edge Kernel

```
conv2d = nn.Conv2d(1, 1, kernel_size=(1, 2), bias=False)

X = X.reshape((1, 1, 6, 8))
Y = Y.reshape((1, 1, 6, 7))
lr = 0.03

for epoch in range(10):
    Y_hat = conv2d(X)
    loss = (Y_hat - Y) ** 2

    conv2d.zero_grad()
    loss.sum().backward()
    conv2d.weight.data[:] -= lr * conv2d.weight.grad
```

관찰

랜덤하게 시작한 kernel이 학습 후 $[1, -1]$ 에 가까워진다. CNN의 kernel은 사람이 정하는 규칙이 아니라 데이터에서 배우는 규칙이다.

7. Feature Map과 Receptive Field

- **Feature map:** 한 kernel이 입력 전체에서 만든 반응 지도
- **Receptive field:** 출력의 한 값이 실제로 바라보는 입력 영역
- layer가 깊어질수록 receptive field가 넓어진다.

좋은 해석

낮은 layer는 좁은 영역의 edge를 보고, 높은 layer는 넓은 영역의 object pattern을 본다.

$$r_{\ell} = r_{\ell-1} + (k_{\ell} - 1)$$

stride가 1일 때

k_{ℓ} 은 ℓ 번째 layer의 kernel size이다.

Part 3. 크기 제어: Padding과 Stride

1. Convolution을 하면 왜 작아지는가?

- kernel이 이미지 바깥으로 나갈 수 없기 때문에 가장자리는 계산 위치가 줄어든다.
- 입력 크기가 $n_h \times n_w$, kernel 크기가 $k_h \times k_w$ 이면 출력은 작아진다.

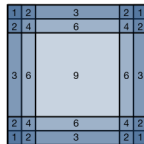
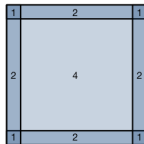
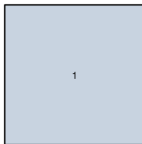
$$(n_h - k_h + 1) \times (n_w - k_w + 1)$$

문제

여러 번 convolution을 쌓으면 공간 크기가 계속 줄고, 가장자리 정보가 빠르게 사라질 수 있다.

2. Padding: 가장자리를 보호하기

- Padding은 입력 가장자리에 추가 픽셀을 붙인다.
- 보통 0으로 채우는 zero padding을 사용한다.
- kernel 크기가 홀수이면 적절한 padding으로 입력과 출력 크기를 같게 만들 수 있다.



3. Padding이 들어간 출력 크기

$$(n_h - k_h + 2p_h + 1) \times (n_w - k_w + 2p_w + 1)$$

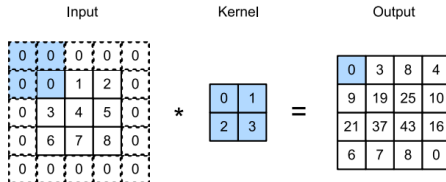
예시

입력: 8×8

kernel: 3×3

padding: 1

출력: 8×8



4. PyTorch에서 Padding

Padding Example

```
import torch
from torch import nn

conv2d = nn.Conv2d(1, 1, kernel_size=3, padding=1)

X = torch.rand(8, 8)
X = X.reshape((1, 1, 8, 8))
Y = conv2d(X)

print(Y.shape)  # torch.Size([1, 1, 8, 8])
```

기억할 문장

Padding은 convolution을 반복해도 공간 크기를 유지하게 해준다.

5. Stride: 얼마나 건너뛰며 볼 것인가

- Stride는 kernel이 이동하는 간격이다.
- stride가 커지면 출력 크기가 줄어든다.
- 계산량을 줄이고 큰 구조를 빠르게 요약한다.

직관

stride 1은 모든 위치를 보고, stride 2는 한 칸씩 건너뛰며 본다.

Input Kernel Output

0	0	0	0	0
0	0	1	2	0
0	3	4	5	0
0	6	7	8	0
0	0	0	0	0

*

0	1
2	3

=

0	8
6	8

6. Stride가 들어간 출력 크기

$$\left\lfloor \frac{n_h - k_h + p_h + s_h}{s_h} \right\rfloor \times \left\lfloor \frac{n_w - k_w + p_w + s_w}{s_w} \right\rfloor$$

Padding과 Stride의 역할 비교

개념	주요 효과	수업용 한 줄
Padding	공간 크기 유지, 가장자리 활용 증가	줄어드는 것을 막는다.
Stride	공간 크기 감소, 계산량 감소	더 듬성듬성 샘플링한다.

Part 4. Channels and Pooling

1. Multiple Input Channels

- 실제 이미지는 보통 RGB처럼 여러 channel을 가진다.
- 입력 channel이 c_i 개라면 kernel도 c_i 개의 channel을 가져야 한다.
- 각 channel별 convolution 결과를 더해서 하나의 feature map을 만든다.

$$X \in \mathbb{R}^{c_i \times h \times w}, \quad K \in \mathbb{R}^{c_i \times k_h \times k_w}$$

$$Y = \sum_{c=1}^{c_i} (X_c * K_c)$$

2. Multi-Channel Convolution 코드

Multiple Input Channels

```
def corr2d_multi_in(X, K):  
    return sum(corr2d(x, k) for x, k in zip(X, K))  
  
X = torch.tensor([  
    [[0., 1., 2.], [3., 4., 5.], [6., 7., 8.]],  
    [[1., 2., 3.], [4., 5., 6.], [7., 8., 9.]]  
)  
  
K = torch.tensor([  
    [[0., 1.], [2., 3.]],  
    [[1., 2.], [3., 4.]]  
)  
  
print(corr2d_multi_in(X, K))
```

해석

각 channel은 서로 다른 정보를 담고, CNN은 channel별 특징을 추출한 뒤 합친다.

3. Pooling이 필요한 이유

- CNN의 최종 목표는 이미지 전체에 대한 판단이다.
- 따라서 점점 넓은 영역의 정보를 요약해야 한다.
- Pooling은 공간 크기를 줄이면서 중요한 정보를 유지한다.

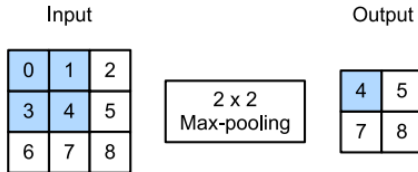
위치 변화에 대한 강건성

Convolution은 “이 위치에 edge가 있다”에 가깝고, pooling은 “이 근처에 edge가 있다”에 가깝다.

4. Max Pooling과 Average Pooling

- Max pooling: window 안의 가장 큰 값을 선택한다.
- Average pooling: window 안의 평균값을 계산한다.
- 학습되는 parameter는 없다.

$$\begin{bmatrix} 0 & 1 \\ 3 & 4 \end{bmatrix} \rightarrow 4$$



5. Pooling 직접 구현

Simple 2D Pooling

```
def pool2d(X, pool_size, mode='max'):  
    p_h, p_w = pool_size  
    Y = torch.zeros((X.shape[0] - p_h + 1,  
                    X.shape[1] - p_w + 1))  
  
    for i in range(Y.shape[0]):  
        for j in range(Y.shape[1]):  
            patch = X[i:i+p_h, j:j+p_w]  
            if mode == 'max':  
                Y[i, j] = patch.max()  
            else:  
                Y[i, j] = patch.mean()  
  
    return Y
```

Convolution과의 차이

Pooling은 kernel weight를 학습하지 않고, 정해진 규칙으로 요약만 수행한다.

6. Pooling과 Channel

- Pooling은 convolution과 달리 channel을 합치지 않는다.
- 각 channel에 독립적으로 pooling을 적용한다.
- 따라서 입력 channel 수와 출력 channel 수는 같다.

정리

Convolution은 channel 정보를 섞어 새로운 feature map을 만들 수 있고, pooling은 각 feature map의 공간 크기만 줄인다.

Part 5. LeNet

1. CNN의 두 블록

Feature Extraction

Convolution, activation, pooling을 반복하며 이미지를 feature tensor로 변환한다.

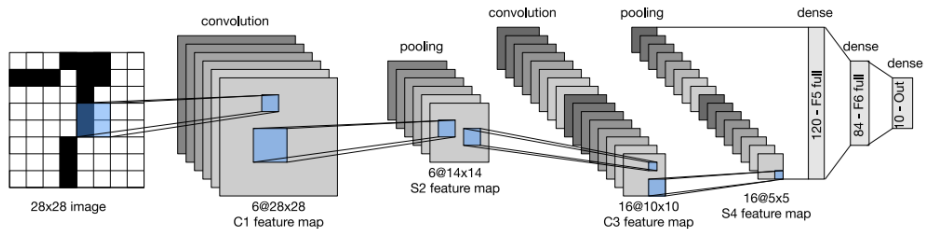
Classification

Flatten 후 fully connected layer로 class score를 계산한다.

전체 관점

앞부분은 “무엇이 보이는가”를 만들고, 뒷부분은 “그래서 어떤 class인가”를 결정한다.

2. LeNet 구조



3. LeNet의 데이터 흐름

Layer-wise Transformation

단계	출력 형태	역할
Input	$1 \times 28 \times 28$	흑백 이미지
Conv1 + Pool	$6 \times 14 \times 14$	단순 특징 추출 및 축소
Conv2 + Pool	$16 \times 5 \times 5$	더 복잡한 특징 추출
Flatten	400	dense layer 입력으로 변환
Dense	10	class score 출력

4. Convolutional Block의 역할

- **Convolution:** edge, texture, shape 같은 국소 특징을 찾는다.
- **Activation:** 비선형성을 추가해 복잡한 패턴을 표현한다.
- **Pooling:** feature를 요약하고 위치 변화에 덜 민감하게 만든다.

LeNet과 현대 CNN

원래 LeNet은 sigmoid를 사용했지만, 현대적인 구현에서는 학습 속도와 gradient 문제 때문에 ReLU를 자주 사용한다.

5. Dense Block의 역할

- Convolution block의 출력은 (*batch, channel, height, width*) 형태이다.
- Dense layer에 넣으려면 먼저 flatten해야 한다.
- LeNet에서는 $16 \times 5 \times 5 = 400$ 차원 feature vector가 된다.

$$(batch, 16, 5, 5) \rightarrow (batch, 400) \rightarrow (batch, 10)$$

분류의 의미

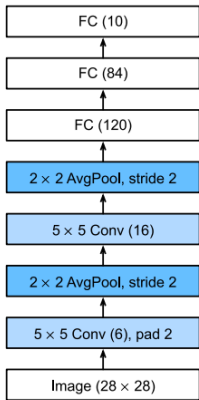
마지막 10개 값은 Fashion-MNIST의 10개 class에 대한 score이다.

6. PyTorch로 구현한 LeNet

LeNet Model

```
model = nn.Sequential(  
    nn.Conv2d(1, 6, kernel_size=5, padding=2),  
    nn.ReLU(),  
    nn.AvgPool2d(kernel_size=2, stride=2),  
  
    nn.Conv2d(6, 16, kernel_size=5),  
    nn.ReLU(),  
    nn.AvgPool2d(kernel_size=2, stride=2),  
  
    nn.Flatten(),  
    nn.Linear(16*5*5, 120),  
    nn.ReLU(),  
    nn.Linear(120, 84),  
    nn.ReLU(),  
    nn.Linear(84, 10)  
)
```


7. Layer별 출력 확인



- 공간 크기는 점점 감소한다.
- channel 수는 증가한다.
- 마지막에는 class score만 남는다.

핵심 변화

이미지 → feature maps → feature vector → class scores

Part 6. lenet_experiment.py

1. 실험 코드의 목적

Experiment Configurations

```
configs = [  
    (120, 84),  
    (256, 128),  
    (128, 64),  
    (64, 32),  
    (400, 200)  
]
```

- 같은 LeNet feature extractor를 사용한다.
- Dense block의 hidden size인 fc1, fc2만 바꾼다.
- Fashion-MNIST에서 어떤 dense 구조가 더 좋은 정확도를 내는지 비교한다.

2. 데이터 준비

Fashion-MNIST DataLoader

```
transform = transforms.ToTensor()

train_dataset = datasets.FashionMNIST(
    root='./data', train=True, download=True, transform=transform)

test_dataset = datasets.FashionMNIST(
    root='./data', train=False, download=True, transform=transform)

train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=128)
```

해석

이미지를 tensor로 바꾸고, mini-batch 단위로 학습과 평가를 수행할 준비를 한다.

3. 모델 정의: LeNet 클래스

LeNet Constructor

```
class LeNet(nn.Module):
    def __init__(self, fc1=120, fc2=84):
        super().__init__()

        self.conv = nn.Sequential(
            nn.Conv2d(1, 6, kernel_size=5, padding=2),
            nn.ReLU(),
            nn.AvgPool2d(2),
            nn.Conv2d(6, 16, kernel_size=5),
            nn.ReLU(),
            nn.AvgPool2d(2)
        )
```

핵심

Convolution block은 고정되어 있고, 입력 이미지를 $16 \times 5 \times 5$ feature map으로 바꾼다.

4. Dense Block: 실험 대상

Classification Head

```
self.fc = nn.Sequential(  
    nn.Flatten(),  
    nn.Linear(16*5*5, fc1),  
    nn.ReLU(),  
    nn.Linear(fc1, fc2),  
    nn.ReLU(),  
    nn.Linear(fc2, 10)  
)
```

무엇을 비교하는가?

fc1, fc2가 커지면 표현력은 증가하지만 parameter와 overfitting 위험도 함께 증가한다.

5. 학습 함수

Training Loop

```
def train(model, loader, optimizer, criterion, device):  
    model.train()  
    total_loss = 0  
  
    for X, y in loader:  
        X, y = X.to(device), y.to(device)  
        optimizer.zero_grad()  
        out = model(X)  
        loss = criterion(out, y)  
        loss.backward()  
        optimizer.step()  
        total_loss += loss.item()  
  
    return total_loss / len(loader)
```

6. 평가 함수

Evaluation Loop

```
def test(model, loader, device):
    model.eval()
    correct = 0
    total = 0

    with torch.no_grad():
        for X, y in loader:
            X, y = X.to(device), y.to(device)
            out = model(X)
            preds = out.argmax(dim=1)
            correct += (preds == y).sum().item()
            total += y.size(0)

    return correct / total
```

평가에서 중요한 점

`model.eval()`과 `torch.no_grad()`로 dropout/batchnorm 모드와 gradient 계산을 평가용으로 바꾼다.

7. 실험 실행

Run One Configuration

```
def run_experiment(fc1, fc2, device):  
    print(f"\nRunning: FC1={fc1}, FC2={fc2}")  
    model = LeNet(fc1, fc2).to(device)  
  
    optimizer = optim.SGD(model.parameters(), lr=0.1)  
    criterion = nn.CrossEntropyLoss()  
  
    for epoch in range(5):  
        loss = train(model, train_loader, optimizer, criterion, device)  
        acc = test(model, test_loader, device)  
        print(f"Epoch {epoch+1}: loss={loss:.4f}, acc={acc:.4f}")  
  
    return acc
```

8. 결과를 볼 때 질문할 것

- Dense layer를 크게 만들수록 accuracy가 항상 좋아지는가?
- 작은 dense layer는 parameter가 적어도 충분한 성능을 내는가?
- 학습 epoch가 5회라면 결과가 안정적이라고 말할 수 있는가?
- SGD learning rate 0.1은 모든 구조에 공정한 설정인가?

수업 마무리 포인트

CNN의 앞부분은 feature를 만들고, dense block은 그 feature를 분류한다. `lenet_experiment.py`는 dense block의 크기가 최종 분류 성능에 어떤 영향을 주는지 실험하는 코드이다.

- CNN은 locality와 weight sharing으로 이미지 구조를 활용한다.
- Kernel은 학습 가능한 feature detector이다.
- Padding은 크기를 유지하고, stride와 pooling은 크기를 줄이며 정보를 요약한다.
- LeNet은 Conv/Pool feature extractor와 Dense classifier를 결합한 고전적인 CNN이다.
- 실험 코드는 dense hidden size를 바꾸며 LeNet의 classification head를 비교한다.

Takeaway

CNN을 이해한다는 것은 “이미지가 layer를 지날 때 shape와 의미가 어떻게 변하는지”를 추적할 수 있다는 뜻이다.

Q & A